

Designing Classes



Two dimensional arrays

```
double[][] a = new double[3][4];
```

```
int M = 3; int N = 4;
```

```
double[][] ar = new double[M][N];
```

```
for (int i = 0; i < M; i++)
```

```
{
```

```
    for (int j = 0; j < N; j++)
```

```
    { ar[i][j] = 0.0;
```

```
    }
```

```
}
```

Initialize 2D array using values

```
double[ ][ ] ar = {  
    { .02, .92, .02, .07, .02 },  
    { .02, .03, .32, .32, .32 },  
    { .04, .02, .02, .92, .08 },  
    { .92, .02, .02, .04, .02 },  
    { .47, .02, .47, .02, .01 },  
};
```

Matrix addition

```
double[ ][ ] c = new double[N][N];  
for (int i = 0; i < N; i++)  
    for (int j = 0; j < N; j++)  
        c[i][j] = a[i][j] + b[i][j];
```

The `?:` Operator

- In addition to the **if** statement, Java provides a more compact way to express conditional execution that can be extremely useful in certain situations.

$$condition \ ? \ expression_1 \ : \ expression_2$$

- When Java evaluates the `?:` operator, it first determines the value of *condition*, which must be a **boolean**. If *condition* is **true**, Java evaluates *expression₁* and if *condition* is **false**, Java evaluates *expression₂*
- You could use the `?:` operator to assign the larger of **x** and **y** to the variable **max** like this:

$$max = (x > y) \ ? \ x \ : \ y;$$

The `switch` Statement

The **`switch`** statement provides a convenient syntax for choosing among a set of possible paths:

```
switch ( expression ) {  
    case  $v_1$ :  
        statements to be executed if expression =  $v_1$   
        break;  
    case  $v_2$ :  
        statements to be executed if expression =  $v_2$   
        break;  
    ... more case clauses if needed ...  
    default:  
        statements to be executed if no values match  
        break;  
}
```

Command-Line Input vs. Standard Input

Command-line inputs

- Use command-line inputs to read a few user values
- Input entered before program begins execution

Standard input

- Flexible OS abstraction for input
- Input entered while program is executing

Standard input.

- **StdIn** is library for reading text input.

```
public class StdIn
```

boolean	isEmpty()	<i>true if no more values, false otherwise</i>
int	readInt()	<i>read a value of type int</i>
double	readDouble()	<i>read a value of type double</i>
long	readLong()	<i>read a value of type long</i>
boolean	readBoolean()	<i>read a value of type boolean</i>
char	readChar()	<i>read a value of type char</i>
String	readString()	<i>read a value of type String</i>
String	readLine()	<i>read the rest of the line</i>
String	readAll()	<i>read the rest of the text</i>

Standard output

- **StdOut** is library for writing text output

```
public class StdOut
```

```
void print(String s)
```

print s

```
void println(String s)
```

print s, followed by newline

```
void println()
```

print a new line

```
void printf(String f, ... )
```

formatted print

A example: reading input string

```
import java.io.*;
public class Ball {
    public static void main (String[] args) {
        BufferedReader stdin = new BufferedReader (new
                                                    InputStreamReader(System.in));
        String message;          // Creates a variable called message for input
        System.out.print ("Enter the message : ");

        message = stdin.readLine();
        System.out.println("You entered : " + message);
    } // method main
} // end of class
```

An example: read integer

```
public class Add {  
    public static void main(String[] args) {  
        BufferedReader stdin = new BufferedReader (new  
            InputStreamReader(System.in));  
        System.out.print("Type the first integer: ");  
        String number1 = stdin.readLine();  
        int x = Integer.parseInt(number1);  
        System.out.print("Type the second integer: ");  
        String number2 = stdin.readLine();  
        int y = Integer.parseInt(number2);  
        int sum = x + y;  
        System.out.println("Their sum is " + sum);    } }
```

Another example

```
public class Average {  
    public static void main(String[] args) {  
        BufferedReader stdin = new BufferedReader (new  
            InputStreamReader(System.in));  
        double sum = 0.0; // cumulative total  
        int n = 0;        // number of values  
        while (!stdin.isEmpty()) {  
            // String number = stdin.readLine();  
            // double y = Double.parseDouble(number);  
            double x = stdin.readDouble();  
            sum = sum + x;  
            n++;    }  
        StdOut.println(sum / n);  
    }  
}
```

Ctrl z to exit (Windows)
Ctrl d (Linux,etc)